

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 7

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 16-08-2026

1. TITLE

Package Management in Node.js: NPM, package.json, and External Package Integration

2. AIM

To explain the architecture of Node Package Manager (NPM) and the package.json manifest file, and to demonstrate the step-by-step procedure of initializing a Node.js project, installing an external package (Express), and utilizing it to build an HTTP application.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the purpose of NPM as a package manager and online registry for JavaScript.
 - To explore the structure, purpose, and key attributes of the package.json manifest file.
 - To differentiate between runtime dependencies (dependencies) and development dependencies (devDependencies).
 - To execute standard NPM commands (npm init, npm install) to configure project metadata.
 - To install the external 'express' package and import it in a Node.js server script to handle HTTP routes.
-

4. THEORY

NPM (Node Package Manager)

NPM is the default package manager for the Node.js runtime. It comprises two key components:

1. The NPM Registry: A public online database containing hundreds of thousands of reusable open-source JavaScript packages and libraries.
2. The NPM CLI (Command-Line Interface): A terminal tool used by developers to download, update, configure, and manage project dependencies and automated scripts.

Understanding package.json and Key Files

The package.json file resides at the root of a Node.js project and serves as the project's central manifest. Key fields include:

- name & version: The unique identifier and semantic version (e.g. 1.0.0) of the project.
- main: The primary entry point file of the application (e.g., server.js or index.js).
- scripts: Custom shortcut commands (e.g., "start": "node server.js").
- dependencies: External packages required for the application to run in production (e.g., express).
- devDependencies: Packages needed only during development and testing (e.g., nodemon, jest).
- node_modules/ folder: Directory where downloaded package code and transitive dependencies are stored.
- package-lock.json: Automatically generated file that records the exact versions and integrity hashes of installed packages to ensure deterministic builds.

Steps to Install and Use an External Package

Step 1 - Initialize Project: Run 'npm init -y' in the terminal. This generates a default package.json file.

Step 2 - Install External Package: Run 'npm install express'. NPM fetches Express from the registry, places it in node_modules, and adds "express": "^4.19.2" to dependencies in package.json.

Step 3 - Import Package in Application: In server.js, require the installed package via `const express = require("express")`.

Step 4 - Utilize Package Features: Create the express app, define routes (e.g. `app.get("/", ...)`), and start listening on a port.

Step 5 - Execute Application: Run `node server.js` and test the endpoints in browser.

5. ALGORITHM / PROCEDURE

- Open the project working directory in terminal.
- Initialize a new Node.js project by executing: `npm init -y`.
- Verify that package.json is created in the root directory.
- Install Express by executing: `npm install express`.
- Verify that 'express' appears under the 'dependencies' section of package.json and that node_modules/ and package-lock.json are created.
- Create a server script named appServer.js.
- Import express using `const express = require("express")`.
- Initialize the app: `const app = express()`.
- Configure root route GET '/' to send "Welcome to my Node.js application!".
- Configure about route GET '/about' to send "This is the About Page."
- Bind the server to port 3000 using `app.listen(3000, ...)`.
- Start the server with `node appServer.js` and verify HTTP responses.

6. PROGRAM

1. package.json

```
{  
  
  "name": "pbl-7",  
  "version": "1.0.0",  
  "description": "",  
  "main": "npm.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs",  
  "dependencies": {  
    "express": "^5.2.1"  
  }  
}
```

2. appServer.js

```
// Import the Express package installed via NPM  
const express = require("express");  
  
// Create an Express application  
const app = express();  
  
// Define a route for the home page  
app.get("/", (req, res) => {  
  // Send a response to the browser  
  res.send("Welcome to my Node.js application!");  
});  
  
// Define another route  
app.get("/about", (req, res) => {  
  // Send response for the About page  
  res.send("This is the About Page.");  
});  
  
// Start the server on port 3000  
app.listen(3000, () => {
```

```
    console.log("Server running at http://localhost:3000");  
  });
```

7. CODE EXPLANATION

Code / Component	Explanation
<code>npm init -y</code>	Initializes a Node project with default configuration and generates package.json.
<code>npm install express</code>	Downloads Express package and records it in package.json dependencies.
<code>"dependencies": { "express": "..." }</code>	Declares that the application relies on Express for runtime execution.
<code>const express = require("express")</code>	Resolves and loads the express package from the node_modules folder.
<code>app.get("/", (req, res) => ...)</code>	Registers an HTTP GET route handler on the root endpoint.
<code>res.send("...")</code>	Transmits HTTP response text to the requesting client.
<code>app.listen(3000, ...)</code>	Starts the HTTP server listening on TCP port 3000.

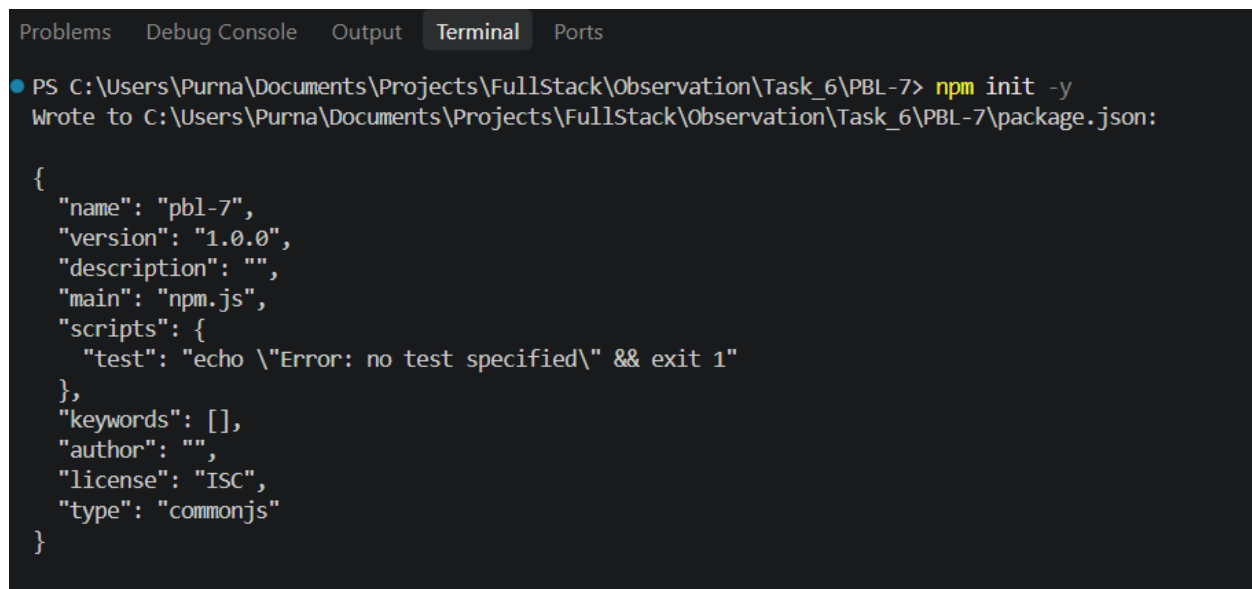
8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Execute npm init -y	package.json created successfully with project metadata	Manifest generated	Pass
TC-02	Execute npm install express	Express installed into node_modules, added to package.json	Dependency recorded	Pass
TC-03	Start server via node appServer.js	Server boots up and logs "Server running at http://localhost:3000"	Server running on port 3000	Pass
TC-04	Send GET request to	Returns 200 OK with body	Response text received	Pass

	http://localhost:3000/	"Welcome to my Node.js application!"		
TC-05	Send GET request to http://localhost:3000/about	Returns 200 OK with body "This is the About Page."	About response received	Pass

9. OUTPUT

Output 1: NPM Initialization in Terminal



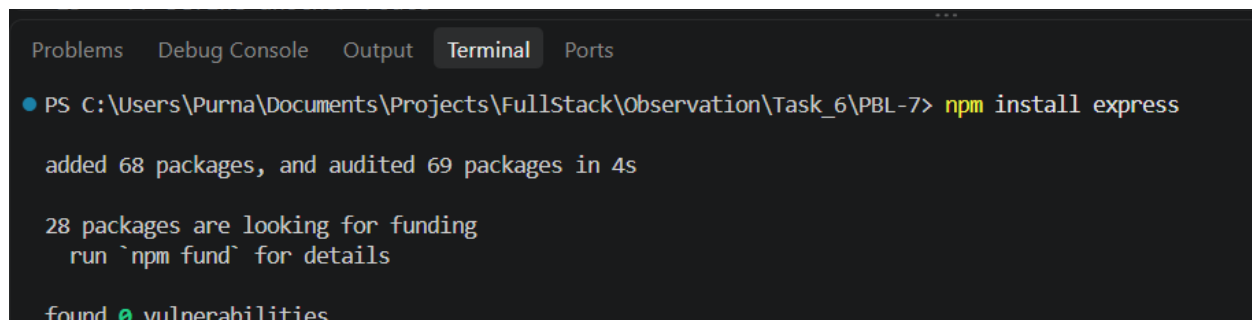
```

Problems  Debug Console  Output  Terminal  Ports
● PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-7> npm init -y
Wrote to C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-7\package.json:

{
  "name": "pbl-7",
  "version": "1.0.0",
  "description": "",
  "main": "npm.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

```

Output 2: Express Installation in Terminal



```

Problems  Debug Console  Output  Terminal  Ports
● PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-7> npm install express

added 68 packages, and audited 69 packages in 4s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

Output 3: Server Execution

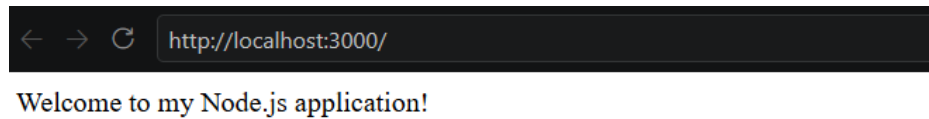
Terminal output of Server Execution.

```
Problems  Debug Console  Output  Terminal  Ports

PS C:\Users\Purna\Documents\Projects\FullStack\Observation\Task_6\PBL-7> node npm.js
Server running at http://localhost:3000
█
```

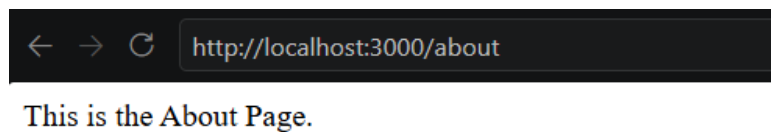
Output 4:Home Route

HTTP responses received from the Express application of Home Route.



Output 5:About Route

HTTP responses received from the Express application of About Route.



10. RESULT

Thus, the concepts of NPM and package.json in Node.js were thoroughly explained and demonstrated. An external package (Express) was successfully installed and utilized within a Node.js web application to handle HTTP routes. All configuration and execution test cases passed successfully.